

SafeExec Artifact Hardening and Reproducibility Check

AI-use disclosure. Drafted with Codex (GPT-5, Codex desktop app) from student-authored project artifacts and command outputs generated in this repository. AI-drafted, student-revised. Full audit trail: `docs/ai-use-log.md`.

Course: CISC 699 Applied Project in Computer Information Sciences

Student: Zixuan Liang

Advisor: Prof. Khalid Lateef

Instructor: Dr. Majid Shaalan

Assignment: 10 Hard Stop 5: Artifact Hardening and Reproducibility Check

Prepared: 2026-06-26

1. Hardening Question

The main W10 question is:

Can another technical reader receive the SafeExec artifact, identify the runtime assumptions, run the current prototype, execute the validation path, and understand what remains environment-dependent without needing private context from the student?

The W8 package showed that the execution path is technically alive. W10 focuses on transferability: setup instructions, environment files, packaging, documentation completeness, and clean-run evidence.

2. What Was Hardened

- **Setup entry point:** added `.env.example` and expanded README setup commands. Evidence: `.env.example`, `README.md`.
- **Reproducibility docs:** added runbook, environment/configuration note, data/redistribution note, and artifact manifest. Evidence: `docs/reproducibility/`.
- **Audit path:** added `scripts/audit_reproducibility.py` and

`make repro-audit`. Evidence: `evidence/reproducibility-audit.md`.

- **Packaging path:** added `scripts/package_artifact.py` and

`make package-artifact`. Evidence:

`safeexec-reproducibility-package.tar.gz`.

- **Clean-run proof:** built a tarball, unpacked it under `/tmp`, and ran smoke, tests, validation, and audit. Evidence:

`evidence/clean-run-output.txt`.

- **User orientation:** added Canvas checklist and walkthrough script. Evidence:

`canvas-submission-checklist.md`, `walkthrough-script.md`.

3. Current Artifact Package

The handoff package is:

`docs/10-hard-stop-5/safeexec-reproducibility-package.tar.gz`

The package excludes `.git`, Python caches, compiled Python files, ZIP files, and generated package archives. It includes:

- Source code under `src/safeexec/`.
- Functional tests under `tests/functional/`.
- Validation scripts under `scripts/`.
- Make targets for smoke, test, validation, audit, and packaging.
- Documentation under `README.md`, `deploy/README.md`, and

`docs/reproducibility/`.

- Course evidence docs and AI-use/process logs.

4. Setup and Execution Path

Reviewer setup path:

```
git clone https://github.com/liangzixuan/cisc-699.git
cd cisc-699
python3 --version
cp .env.example .env
make smoke
make test
make validate
make repro-audit
```

Equivalent direct commands are provided in

`docs/reproducibility/runbook.md` for environments without make.

Container checks remain Linux-target-host checks:

```
docker pull python:3.11-slim
PYTHONPATH=src python3 scripts/run_validation_workflow.py --output-dir docs/10-hard-stop-5/evidence-target --repeat 3 --include-docker
```

The local backend remains a development smoke path only. Docker and gVisor are the containment-relevant paths.

5. Clean-Run Evidence

The W10 clean run used the package builder rather than the live working tree:

1. Built `/tmp/safeexec-w10-clean-package.tar.gz`.
2. Unpacked it into a fresh `/tmp/safeexec-w10-clean-*` directory.
3. Copied `.env.example` to `.env`.
4. Ran `make smoke`.
5. Ran `make test`.
6. Ran local validation with `scripts/run_validation_workflow.py`.
7. Ran `scripts/audit_reproducibility.py`.

Clean-run output summary:

Check	Result	Evidence
Smoke execution	Passed; returned structured local result with <code>status: ok</code> .	<code>evidence/clean-run-output.txt</code>
Functional tests	Passed; Ran 7 tests ... OK.	<code>evidence/clean-run-output.txt</code> , <code>evidence/test-output.txt</code>
Local validation	Passed; local 12/12 and API trace passed.	<code>evidence/validation-summary.txt</code>
Reproducibility audit	Passed; required files, README markers, Make targets, and dependency policy present.	<code>evidence/reproducibility-audit.md</code>

The local validation result is:

```
include_docker: False
all_passed: True
Backend summary:
- local: 12/12 passed
- api_trace: passed
```

6. Environment and Dependency Status

Current dependency policy is intentionally simple:

- `pyproject.toml` declares Python `>=3.11`.

- `requirements.txt` states that the current prototype uses only the Python standard library.
- `.env.example` documents configuration knobs and explicitly contains no secrets.

Local evidence was captured on macOS with Python 3.14.5. The prior Docker and gVisor evidence remains target-host evidence because local macOS Docker is not available in the authoring environment. The Docker/gVisor environment is documented in W8 evidence and in `docs/reproducibility/environment.md`.

7. Data and Redistribution

SafeExec currently redistributes no third-party benchmark dataset. The committed validation and midpoint cases are student-authored Python snippets. The future HumanEval/MBPP work must use task-ID manifests or preserve upstream license notices if any prompts or expected outputs are copied into the repository.

The current policy is documented in:

`docs/reproducibility/data-and-redistribution.md`

8. Friction Points and Fixes

Friction point	W10 handling	Remaining action
README previously described the project well but did not provide a single reviewer setup path.	Added fresh setup commands and W10 section.	Keep current as commands evolve.
Configuration assumptions were implicit.	Added <code>.env.example</code> .	Load <code>.env</code> automatically only if/when config expands.
Artifact contents were spread across several milestone folders.	Added <code>artifact-manifest.md</code> .	Add final report/deck entries later.
No dedicated reproducibility audit command existed.	Added <code>make repro-audit</code> .	Expand audit to fail on stale generated evidence before final submission.
Source package creation was manual.	Added <code>make package-artifact</code> .	Rebuild final source package after last code/report changes.
Docker/gVisor cannot be reproduced on the local authoring machine.	Documented target-host path and limitation.	Keep Ubuntu target host as official container benchmark environment.

9. Link to Rubric

- **Artifact completeness and functionality:** source, tests, validation scripts, API shell, and benchmark scripts are included; smoke/test/validation pass.
- **Reproducibility and setup quality:** `.env.example`, runbook, environment note, Make targets, audit script, and clean-run transcript.
- **Quality assurance and hardening:** `make test`, local validation, audit script, package script, and issue notes.
- **README, packaging, and user orientation:** README W10 section, docs/reproducibility/, source tarball, Canvas checklist, walkthrough script.
- **Recorded walkthrough support:** `walkthrough-script.md` gives a 5-7 minute recording path.
- **AI usage and academic integrity:** updated docs/ai-use-log.md and inline disclosure blocks.

10. Next Hardening Steps

Before final submission, SafeExec still needs:

1. A host-side Python 3.11 validation environment on the Ubuntu target host or a dev container that removes host-Python ambiguity.
2. A clearer Docker/gVisor cold-start versus warm-run benchmark protocol.
3. HumanEval/MBPP corpus manifest with license/provenance notes.
4. Expanded adversarial suite with per-program expected contained outcomes.
5. Final report appendices that link every result table to the exact command and environment that produced it.